

Week 2

Multilayer Perceptron and its Hyper parameter Tuning

Multilayer Perceptron (MLP) Overview

- ❖ A Multilayer Perceptron (MLP) is a type of artificial neural network.
- ❖ It consists of multiple layers of nodes, each layer fully connected to the next one. The primary purpose of an MLP is to learn complex patterns in data and perform tasks like classification, regression, and more.
- ❖ **Key Components:**
 - ❖ **Input Layer:** Receives input features.
 - ❖ **Hidden Layers:** Intermediate layers where computations occur. The number of layers and neurons in each layer can be tuned.
 - ❖ **Output Layer:** Produces the final prediction.

Hyperparameters in MLP:

- ❖ **Number of hidden layers:** Number of layers between input and output layers.
- ❖ **Number of neurons per layer:** Determines the capacity of the model.
- ❖ **Learning rate:** Controls how much the model adjusts its weights with each iteration.
- ❖ **Activation function:** Introduces non-linearity into the network (e.g., ReLU, sigmoid, tanh).
- ❖ **Batch size:** Number of samples the model processes before updating weights.
- ❖ **Number of epochs:** The number of times the entire dataset passes through the network.
- ❖ **Optimizer:** Algorithm used to minimize the loss function (e.g., Adam, SGD).

Hyperparameter Tuning:

- ❖ Hyperparameters are configuration settings that control the learning process of an MLP, such as the number of hidden layers, the number of neurons in each layer, the learning rate, and activation functions.
- ❖ Hyperparameter tuning is the process of finding the best combination of hyperparameters to optimize the model's performance.

Hyperparameter Tuning Techniques:

- ❖ **Grid Search:** Exhaustively searches a predefined set of hyperparameter combinations and evaluates the model's performance for each combination.
- ❖ **Random Search:** Randomly samples hyperparameter combinations from predefined ranges, making it computationally efficient compared to grid search.
- ❖ **Bayesian Optimization:** Uses probabilistic models to guide the search for optimal hyperparameters efficiently.
- ❖ **Cross-Validation:** Evaluates the model's performance on different subsets of the training data to avoid overfitting during hyperparameter tuning.

- ❖ Implementing a Multilayer Perceptron (MLP) and tuning its hyperparameters step by step.
- ❖ We will use Python and the tensorflow and keras-tuner libraries.
- ❖ **Step 1:** Install Necessary Libraries Before writing any code, ensure you have the required libraries installed. You can do this using pip:

- 1. pip install tensorflow keras-tuner**

❖ **Step 2: Import Required Libraries** Start by importing the necessary libraries in your Python script.

1. `import tensorflow as tf`
2. `from tensorflow.keras.models import Sequential`
3. `from tensorflow.keras.layers import Dense`
4. `from tensorflow.keras.optimizers import Adam`
5. `from tensorflow.keras.callbacks import EarlyStopping`
6. `from keras_tuner import RandomSearch`
7. `from keras_tuner.engine.hyperparameters import HyperParameters`
8. `from sklearn.datasets import make_classification`
9. `from sklearn.model_selection import train_test_split`
10. `from sklearn.preprocessing import StandardScaler`

❖ Step 3: Generate and Prepare Data For this example, we will generate synthetic data using sklearn. Then, we'll split the data into training and testing sets and normalize it.

1. # Generate synthetic data

2. `X, y = make_classification(n_samples=1000, n_features=20, n_classes=2, random_state=42)`

3. # Split the data into training and testing sets

4. `X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)`

5. # Normalize the data (important for neural networks)

6. `scaler = StandardScaler()`

7. `X_train = scaler.fit_transform(X_train)`

8. `X_test = scaler.transform(X_test)`

StandardScaler

- ❖ StandardScaler is a preprocessing technique in scikit-learn used for standardizing features by removing the mean and scaling to unit variance. StandardScaler, a popular preprocessing technique provided by scikit-learn, offers a simple yet effective method for standardizing feature values
- ❖ StandardScaler is a mean-based scaling method. The formula of StandardScaler is $(X_i - X_{\text{mean}}) / X_{\text{std}}$, so it adjusts the mean as a 0. The StandardScaler is vulnerable to outliers because outliers affect the mean.

Keras

- ❖ **Keras is a high-level**, deep learning API developed by Google for implementing neural networks. It is written in Python and is used to make the implementation of neural networks easy. It also supports multiple backend neural network computation.
- ❖ Keras allows us to prototype, research and deploy deep learning models in an intuitive (spontaneous) and streamlined manner. The functional API makes code comprehensible and stylistic, allowing for effective knowledge transfer between scientists on customer team.

- ❖ **input_dim=20:** This defines the input shape for the model. It specifies that this layer expects 20 input features. This is typically only required for the first layer in the model, as subsequent layers will infer their input shape from the previous layer.
- ❖ **Fully connected layer with a tunable number of neurons (between 16 and 128, in steps of 16). The activation function for this layer is ReLU, and it expects 20 input features. This flexibility in the number of neurons allows Keras Tuner to optimize the model by finding the best configuration during training.**

❖ Step 4: Build the MLP Model Create a function that builds the MLP model. This function will accept hyperparameters, allowing us to easily tune them later.

```
1. def build_model(hp):
2.     model = Sequential()
3.     # First hidden layer with tunable number of units
4.     model.add(Dense(units=hp.Int('units_1',      min_value=16,      max_value=128,
        step=16), activation='relu', input_dim=20))
5.     # Second hidden layer with tunable number of units
6.     model.add(Dense(units=hp.Int('units_2',      min_value=16,      max_value=128,
        step=16), activation='relu'))
7.     # Output layer
8.     model.add(Dense(1, activation='sigmoid'))
```

- ❖ **model.add(Dense(units=hp.Int('units_1', min_value=16, max_value=128, step=16), activation='relu', input_dim=20))**
- ❖ **model.add(Dense(...))**: This adds a fully connected (dense) layer to the model. In a dense layer, each neuron receives input from all neurons in the previous layer.
- ❖ **units=hp.Int('units_1', min_value=16, max_value=128, step=16)**:
- ❖ **units**: Specifies the number of neurons in this layer.
- ❖ **hp.Int('units_1', min_value=16, max_value=128, step=16)**:
- ❖ This is where Keras Tuner comes into play. Instead of hardcoding the number of neurons, this uses **hp.Int** to allow the number to be determined dynamically during the hyperparameter tuning process.
- ❖ **'units_1'**: This is the name of the hyperparameter, which can be referenced later.
- ❖ **min_value=16**: The minimum number of neurons that Keras Tuner will try is 16.
- ❖ **max_value=128**: The maximum number of neurons that Keras Tuner will try is 128.
- ❖ **step=16**: The tuner will try different numbers of neurons in increments of 16 (e.g., 16, 32, 48, ... up to 128).

- ❖ **activation='relu'**: Specifies the activation function to use in this layer. The ReLU (Rectified Linear Unit) activation function is commonly used because it helps the model to learn complex patterns while avoiding some issues like the vanishing gradient problem.
- ❖ The **ReLU activation function** works by applying a simple mathematical operation to the input value. **If the input value is greater than or equal to zero, the output is equal to the input value.** **If the input value is negative, the output is zero.** Mathematically, the ReLU function can be represented as: $f(x) = \max(0, x)$
- ❖ **What is vanishing gradient problem?**
- ❖ **Vanishing Gradient Problem** is a phenomenon that occurs during the training of deep neural networks, where the gradients that are used to update the network become extremely small or "vanish" as they are backpropogated from the output layers to the earlier layers.
- ❖ **Gradient Descent** is an optimization algorithm which is commonly-used to train machine learning models and neural networks. It trains machine learning models by minimizing errors between predicted and actual results.

11. # Compile the model with a tunable learning rate

```
12. model.compile(optimizer=Adam(learning_rate=hp.Float('learning_rate',  
    min_value=0.0001, max_value=0.01, sampling='LOG')),  
    loss='binary_crossentropy', metrics=['accuracy'])  
11. return model
```

- ❖ **Optimizer:** Adam - Adaptive Moment Estimation , with a learning rate that is tunable (between 0.0001 and 0.01 on a logarithmic scale) using Keras Tuner.
- ❖ **Loss Function:** Binary cross-entropy, suitable for binary classification tasks.
- ❖ **Metric:** Accuracy, which will be tracked during training to evaluate the model's performance.
- ❖ By using Keras Tuner to adjust the learning rate, the model's performance can be optimized by finding the best learning rate within the specified range.

Step 5: Set Up Hyperparameter Tuning with Keras TunerNow, set up the tuner. We will use RandomSearch to explore different combinations of hyperparameters.

1. # Instantiate the tuner
2. tuner = RandomSearch(
3. build_model, # Pass the model building function
4. objective='val_accuracy', # Optimize for validation accuracy
5. max_trials=10, # Number of different models to try
6. executions_per_trial=2, # Number of times to train each model
7. directory='my_dir', # Directory to save tuning results
8. project_name='mlp_tuning'
9.)

- ❖ **RandomSearch** tuner that will explore different hyperparameter configurations for your MLP model.
- ❖ It will try up to 10 different sets of hyperparameters, with each set being tested twice to ensure reliability.
- ❖ The tuner will aim to maximize validation accuracy, and all results will be saved in the specified directory and project.
- ❖ This setup is essential for finding the best-performing model through hyperparameter tuning.

❖ **Step 6: Train the Model with Hyperparameter TuningSearch for the best hyperparameters by training the model. This step will automatically train the model multiple times with different hyperparameter settings.**

1. # Early stopping to prevent overfitting

2. `early_stopping = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)`

3. # Perform the hyperparameter search

4. `tuner.search(X_train, y_train, validation_split=0.2, epochs=50, batch_size=32, callbacks=[early_stopping])`

❖ **Step 7: Retrieve the Best Model and Evaluate It** After the search is complete, you can retrieve the best model and evaluate it on the test data

- 1. # Get the best model found by the tuner**
2. `best_model = tuner.get_best_models(num_models=1)[0]`
- 3. # Evaluate the best model on the test set**
4. `loss, accuracy = best_model.evaluate(X_test, y_test)`
5. `print(f"Test Accuracy after Tuning: {accuracy * 100:.2f}%")`

Summary / Conclusion

A Multilayer Perceptron (MLP) is a fundamental type of artificial neural network consisting of an input layer, one or more hidden layers, and an output layer. Each neuron in these layers processes inputs using an activation function (like ReLU) and passes the output to the next layer, allowing the network to learn complex patterns. During training, MLPs use backpropagation to adjust weights and biases, optimizing the model to minimize a loss function (such as binary cross-entropy). Commonly used for tasks like classification and regression, MLPs are versatile and powerful, capable of modeling intricate relationships in data through proper tuning of hyperparameters like learning rate, number of layers, and number of neurons.